HirePortal

Smart Internship & Hiring Portal

Daily Internship Documentation — Day 8

Date	Tuesday, May 5, 2026
Intern	Mageswaran G
Project	Smart Internship & Hiring Portal
Mentor	Sandiya
Module	Week 2 — Module 2: User Profile Management (Day 2 of 4)
Day Focus	Resume Upload Backend — Professional Grade Implementation
Final Status	All resume upload APIs complete and tested — Production ready

1. SOD (Start of Day) — Morning Plan

SOD mail sent to Sandiya in the morning with today's plan.

Day 8 Plan Agreed:

#	Task	Priority
1	Install and configure Multer for file handling	High
2	Create uploads/resumes/ folder structure	High
3	Build POST /api/v1/users/upload-resume API	High
4	Add validations — PDF, DOC, DOCX only + 5MB limit	High
5	Store file path in resumeUrl field in User model	High
6	Test resume upload in Thunder Client / curl	High
7	Begin frontend Profile page UI if time permits	Medium
8	Push to GitHub and send EOD	High

2. Environment Setup

2.1 Multer Installation

Multer is a Node.js middleware for handling file uploads. It receives files from the browser and saves them to the server — like a post office for files.

```
cd ~/smart-hiring-portal/backend
npm install multer
```

Result: Added 11 packages successfully. No vulnerabilities found.

2.2 Upload Folder Structure Created

```
mkdir -p uploads/resumes
touch uploads/resumes/.gitkeep
```

.gitkeep is an empty file that tells Git to track the empty folder. Without it, Git ignores empty folders.

2.3 .gitignore Updated

Added rules to prevent actual resume files from being pushed to GitHub (user privacy + storage concerns):

```
uploads/resumes/*.pdf
uploads/resumes/*.PDF
uploads/resumes/*.doc
uploads/resumes/*.docx
```

---

3. What Was Built Today

3.1 uploadMiddleware.js — Multer Configuration

File: backend/middleware/uploadMiddleware.js

This file configures how files are received, named, filtered, and limited.

Feature	Implementation	Why
Storage destination	uploads/resumes/ folder	Organized file storage location
Filename strategy	resume-{userId}-{timestamp}-{randomHex}.ext	Unique — prevents collisions
Random suffix	crypto.randomBytes(6).toString('hex')	Extra uniqueness — 12 hex chars
File size limit	5MB (5 * 1024 * 1024 bytes)	Prevents server storage abuse
MIME type check	application/pdf, msword, openxml	Layer 1 validation
Extension check	.pdf, .doc, .docx	Layer 2 validation
Auto folder creation	fs.mkdirSync with recursive:true	Prevents crash if folder missing
Module export	{ upload, validateFileSignature }	Both usable from other files

3.2 File Signature Validation (Free Virus Scan Alternative)

Every file type has a special byte pattern at the start — called a magic number or file signature.

Hackers can rename virus.exe to resume.pdf to bypass extension checks. File signature validation reads the actual first bytes of the file to verify it is really what it claims to be.

File Type	Signature (Hex)	ASCII
PDF	25504446 (starts with)	%PDF

DOC (old Word)	D0CF11E0	Binary Office format
DOCX (new Word)	504B0304	PK (ZIP format)

3.3 User Model Extension — Resume Metadata

Changed resumeUrl from a simple String to a structured object to store full resume metadata.

Before:

```
resumeUrl: { type: String, default: '' }
```

After:

```
resume: {
  url:          { type: String, default: '' },
  originalName: { type: String, default: '' },
  size:         { type: Number, default: 0  },
  mimeType:     { type: String, default: '' },
  uploadedAt:   { type: Date               }
}
```

3.4 userService.js — uploadResume Function

File: backend/services/v1/userService.js

Professional upload flow with 4 ordered steps:

Step	Action	Why This Order Matters
Step 1	Find existing user in DB	Check user exists before doing anything
Step 2	Upload new file (done by Multer)	New file is ready before DB update
Step 3	Update DB with new resume metadata	DB updated with new file info
Step 4	Delete old file ONLY after DB success	If DB fails, new file is deleted — no orphan files

Key security features in this function:

- Path traversal protection — validates file path stays inside uploads/resumes/
- Async file operations — non-blocking, does not slow down server
- Safe deletion — uses .catch(() => {}) to handle missing file gracefully
- Public URL stored in DB — not filesystem path — frontend can use directly

3.5 userController.js — uploadResume Controller

Thin layer — only handles req/res. Calls userService.

Added file presence check and file signature validation before calling service.

Returns full resume metadata in response including full URL, original name, size, MIME type, and upload timestamp.

3.6 userRoutes.js — Upload Route with RBAC

File: backend/routes/v1/userRoutes.js

Route middleware chain for POST /upload-resume:

#	Middleware	Purpose
1	uploadLimiter	Rate limit: max 20 uploads per 15 minutes
2	verifyToken	Must be logged in — JWT required
3	authorizeRole('candidate')	Only candidates can upload resumes — company/admin blocked
4	upload.single('resume')	Multer processes the file
5	userController.uploadResume	Save to DB and return response

3.7 server.js — Static File Serving

Added one line to make uploaded files accessible via URL:

```
app.use('/uploads', express.static('uploads'));
```

This means: http://localhost:8000/uploads/resumes/filename.pdf works in the browser.

3.8 errorMiddleware.js — Multer Error Handling

Added Multer-specific error handling so file upload errors return the same standard format as all other API errors:

- LIMIT_FILE_SIZE error → 400: File too large. Maximum size is 5MB
- Wrong file type → 400: Only PDF, DOC, and DOCX files are allowed

3.9 config/index.js — Central Configuration Layer

Created a new file that centralizes all environment variables in one place.

Rule: Only config/index.js can read process.env. All other files use config instead.

This makes the app easier to test, maintain, and switch between environments (dev/staging/production).

3.10 API Versioning — v1 Folder Structure

Reorganized all files into versioned folders. This is how real companies structure their backends.

Before structure:

```
routes/authRoutes.js
controllers/authController.js
services/authService.js
```

After structure (v1):

```
routes/v1/authRoutes.js
```

```
controllers/v1/authController.js
services/v1/authService.js
```

Why: When v2 is built in future, v1 keeps working. No breaking changes for existing users.
The URL /api/v1/auth/login stays the same. Structure behind it changes safely.

---

4. Bugs Fixed Today

#	Error	Root Cause	Fix Applied
1	upload.single is not a function	uploadMiddleware exported whole module instead of { upload }	Changed export to { upload, validateFileSignature }
2	Identifier 'upload' already declared	Both { upload } destructure and const upload existed	Removed duplicate import in userRoutes.js
3	config is not defined	Used config.port before importing config	Added const config = require('./config') at top of server.js
4	Cannot find module '../utils/AppError'	Files moved to v1/ subfolder but imports still used 1-level path	Fixed all paths using sed commands — changed ../ to ../../
5	Cannot find module '../middleware/authMiddleware'	Same issue — routes in v1/ need 2 levels up	Used sed to fix all paths in routes/v1/*.js
6	Unexpected end of input errorMiddleware.js	Missing closing bracket } while editing	Replaced entire file with correct complete version
7	returnDocument: 'after' still in userService.js	grep confirmed it existed — ChatGPT was right	Changed to { new: true } which is correct Mongoose syntax
8	req.body.refreshToken crash after logout	req.body is undefined when no body sent	Added optional chaining: req.body?.refreshToken

---

5. Security Architecture — Resume Upload

3-layer security system for file uploads:

Layer	Where	What It Checks	Example Attack Blocked
Layer 1	uploadMiddleware.js — fileFilter	MIME type from HTTP header	Wrong content type rejected
Layer 2	uploadMiddleware.js — fileFilter	File extension from filename	virus.jpg renamed to resume.pdf
Layer 3	userController.js — validateFileSignature	Actual file byte signature	Executable with .pdf extension

Additional security:

- RBAC — authorizeRole('candidate') — only candidates can upload

- Rate limiting — 20 uploads per 15 minutes per IP
- Path traversal protection — resolvedPath.startsWith(allowedDir) check
- Async fs operations — non-blocking, production safe
- Old file deleted only after DB update succeeds — no orphan files

---

6. API Testing — curl Commands

Thunder Client free version does not support file uploads. Used curl (free terminal tool) instead.

Test #	Command / Scenario	Expected	Result
1	Candidate login — get token	200 — accessToken returned	PASS
2	Candidate uploads PDF resume	200 — full URL + metadata returned	PASS
3	Company tries to upload resume	403 — Access denied. Only candidate allowed	PASS
4	Upload with no file	400 — Please upload a file	PASS
5	Re-upload — old file replaced	200 — old deleted, new saved	PASS
6	Upload returns full URL	http://localhost:8000/uploads/resumes/...	PASS

Sample Successful Response


```
{
  "success": true,
  "message": "Resume uploaded successfully",
  "data": {
    "resume": {
      "url": "http://localhost:8000/uploads/resumes/resume-userId-timestamp-
randomhex.pdf",
      "originalName": "Mageswaran_G.pdf",
      "size": 154868,
      "mimeType": "application/pdf",
      "uploadedAt": "2026-05-05T08:47:15.610Z"
    }
  }
}
```


---

7. Key Concepts Learned Today

Concept	Simple Explanation
Multer	Node.js middleware that handles file uploads. Like a post office — receives files and places them correctly

MIME type	File type identifier sent in HTTP header. Example: application/pdf. Can be spoofed by hackers
File signature	First few bytes of every file — unique per format. PDF always starts with %PDF. Cannot be faked
Magic bytes	Another name for file signature. Reading them confirms actual file type regardless of extension
API versioning	Putting APIs in /v1/ folder. When v2 is built, v1 stays working. No breaking changes
Central config	One file (config/index.js) reads process.env. All other files import from config. Cleaner and testable
Path traversal	Attack where hackers use ../../ in filenames to escape safe folder. Protected by resolvedPath check
Rate limiting	Max requests per time window. Upload route: 20 per 15 min. Prevents spam and abuse
Async fs	fs.promises.unlink() instead of fs.unlinkSync(). Does not block server while deleting file
Orphan file	File on disk with no DB record pointing to it. Prevented by safe delete order in upload flow
sed command	Terminal tool to find and replace text in files. Used to fix all import paths after folder restructure
curl -F flag	Sends multipart form data — required for file uploads in terminal testing

---

8. Professional Upgrades Applied Today

These improvements pushed the backend from internship-level to production-grade:

Upgrade	Before	After	Impact
Filename strategy	userId + timestamp	userId + timestamp + randomHex	Zero collision risk
Resume storage	Simple URL string	Structured object (url, name, size, type, date)	Full metadata available
File validation	MIME type only	MIME + extension + signature	Cannot be bypassed
File deletion	Delete before DB update	Delete only after DB success	No orphan files
Error handling	Raw Multer errors	Normalized through errorMiddleware	Consistent API format
Rate limiting	None on upload	20 per 15 min per IP	Prevents abuse
API structure	Flat folders	Versioned v1/ folders	Future-proof
Config	process.env everywhere	Central config/index.js	Testable, maintainable
Static serving	Not configured	app.use('/uploads', express.static)	Frontend can access files
Path traversal	No protection	resolvedPath.startsWith(allowedDir)	Secure file deletion

---

9. Files Changed Today

File	Action	What Changed
backend/middleware/uploadMiddleware.js	Created	Multer config — storage, filter, limits, signature validation
backend/middleware/errorMiddleware.js	Modified	Added Multer error handling — normalized responses
backend/models/User.js	Modified	Changed resumeUrl String to resume object with metadata fields
backend/services/v1/userService.js	Modified	Added uploadResume with safe flow, path protection, async fs
backend/controllers/v1/userController.js	Modified	Added uploadResume — file check + signature + response
backend/routes/v1/userRoutes.js	Modified	Added upload route with uploadLimiter + RBAC + Multer
backend/server.js	Modified	Added static file serving + config import + v1 route paths
backend/config/index.js	Created	Central config — all process.env in one place
backend/uploads/resumes/.gitkeep	Created	Keeps empty folder tracked in Git
backend/.gitignore	Modified	Added rules to exclude actual resume files from GitHub
backend/package.json	Modified	Added multer dependency
All controllers/v1/*.js	Modified	Fixed import paths after v1 folder restructure
All services/v1/*.js	Modified	Fixed import paths after v1 folder restructure
All routes/v1/*.js	Modified	Fixed import paths after v1 folder restructure

10. GitHub Commits Today

Commit Message	Files Changed	What Was Done
Fix: production-grade cookie config, Mongoose deprecation warning	2 files	authController.js + userService.js cookie and Mongoose fixes
Module 2: Resume upload API complete - Multer, RBAC, validation	8 files	uploadMiddleware + upload route + gitignore + package.json
Module 2: Static file serving, old resume cleanup, double file validation	3 files	server.js static + userService cleanup + uploadMiddleware ext check
Fix: Mongoose new:true, public URL for resumeUrl, safe file deletion	1 file	userService.js — correct Mongoose syntax + public URL
Module 2: Multer error handling added to errorMiddleware	1 file	errorMiddleware.js — Multer errors normalized
Security: path traversal protection, async fs, full URL in response	2 files	userService.js + userController.js security upgrades
Professional: versioned API structure v1/, central config, file signature validation	10 files	v1 folders + config/index.js + signature validation

Fix: All v1 import paths corrected after folder restructure	6 files	All require paths fixed using sed commands

11. EOD (End of Day) Summary

Task	Status
Install Multer and configure	Done ✓
Create uploads/resumes/ folder structure	Done ✓
Build POST /api/v1/users/upload-resume API	Done ✓
File type validation — MIME + extension + signature	Done ✓
5MB file size limit enforced	Done ✓
RBAC — candidates only can upload	Done ✓
Unique filename with userId + timestamp + randomHex	Done ✓
File metadata stored in MongoDB (structured object)	Done ✓
Public URL returned in API response	Done ✓
Static file serving configured in server.js	Done ✓
Old file deleted safely after DB update	Done ✓
Path traversal protection added	Done ✓
Async file operations (fs.promises)	Done ✓
Upload rate limiting (20/15min)	Done ✓
Multer errors normalized in errorMiddleware	Done ✓
Central config/index.js created	Done ✓
API versioned into v1/ folder structure	Done ✓
All import paths fixed after restructure	Done ✓
All 5 curl tests passing	Done ✓
GitHub pushed with clean commits	Done ✓
EOD mail sent to Sandiya	Done ✓

12. Important Notes for Future Reference

- Multer file upload requires Form body type in testing tool — not JSON
- Thunder Client free version does NOT support file uploads — use curl instead
- File signature check reads actual bytes — cannot be faked unlike MIME type
- PDF signature starts with %PDF = hex 25504446
- DOCX is a ZIP file — signature starts with PK = hex 504B0304
- DOC old format signature is D0CF11E0

- Delete old file ONLY after DB update succeeds — prevents orphan files
- Path traversal: always use path.resolve and check startsWith(allowedDir)
- sed -i '' on Mac — the '' is required (Linux uses -i without '')
- v1 folder — routes need ../../ (2 levels up), services need ../../ (2 levels up)
- Central config rule: only config/index.js reads process.env directly
- uploadLimiter separate from authLimiter — upload allows more requests (20 vs 10)
- curl -F flag sends multipart form data required for file uploads
- crypto.randomBytes(6).toString('hex') generates 12 character random hex string

---

13. Tomorrow — Day 9 Plan

Module 2 Frontend — Profile Page UI

#	Task	API to Connect
1	Build ProfilePage.jsx — view mode showing all profile fields	GET /api/v1/users/profile
2	Build edit profile form — bio, location, phone, skills, education	PUT /api/v1/users/profile
3	Build resume upload UI — file input, progress, success message	POST /api/v1/users/upload-resume
4	Show resume download link after upload	Static URL from response
5	Company profile fields — companyName, website, industry	PUT /api/v1/users/profile
6	Add Profile link to dashboard	React Router navigate
7	Test full flow in browser — view, edit, upload	All above
8	Push to GitHub + send EOD	—